UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN
FACULTAD DE CIENCIAS FÍSICO MATEMÁTICAS

ALGORITMIA Y OPTIMIZACIÓN

Reporte Final

Docente: Jose Anastacio Hernandez Saldaña

Alumno: Eduardo Rodríguez Ramos
Matricula: 2116554
Grupo: 031

Semestre Agosto – Diciembre 2026

Ejercicio 1: Aceleración paralela (Parallel Speedup)

Indicación. Ejecutar estimate_pi con 1, 2, 4 y 8 procesos y graficar el speedup contra el número de procesos, con la línea de speedup ideal.

Se lanzan 8 corridas independientes de Monte Carlo (una semilla por corrida, 1,000,000 de muestras cada una) en un Pool de multiprocessing con p procesos. Se toma el mejor tiempo de pared de 3 repeticiones y se calcula S(p) = T1 / Tp y la eficiencia S(p) / p.

Resultados

Procesos	Tiempo (s)	Speedup	Ideal	Eficiencia
1	3.679	1.00	1	100 %
2	2.291	1.61	2	80 %
4	1.598	2.30	4	58 %
8	1.537	2.39	8	30 %

Tabla 1. Tiempos y speedup medidos (equipo de medición con 12 núcleos).

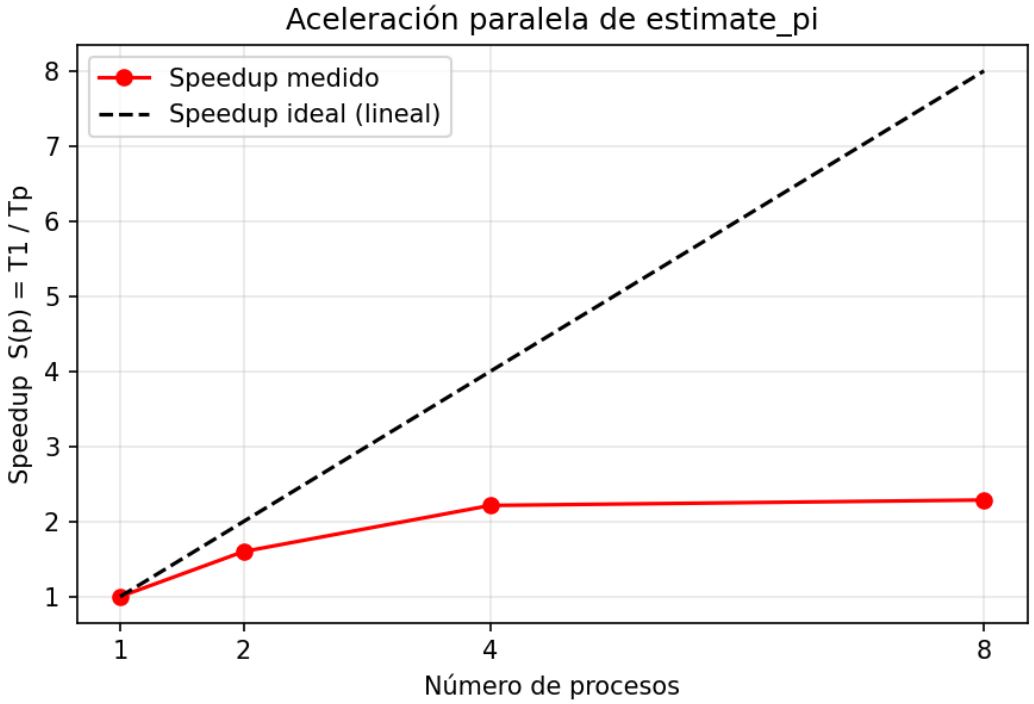


Figura 1. Speedup medido y speedup ideal contra el número de procesos.

Análisis

Los resultados muestran que al aumentar el número de procesos, el programa se ejecuta más rápido, pero la aceleración no es perfectamente lineal. Con 2 procesos, la aceleración es de 1.61 veces; con 4 procesos, aumenta a 2.30 veces, y con 8 procesos llega a 2.39 veces. Esto significa que, aunque utilizar más procesos mejora el rendimiento, el beneficio disminuye a medida que se agregan más. Por ejemplo, pasar de 4 a 8 procesos produce una mejora muy pequeña en comparación con el aumento anterior.

Una de las razones es que trabajar con varios procesos genera sobrecostos de comunicación y sincronización. Los procesos deben repartirse el trabajo, coordinarse y reunir los resultados al finalizar. Estas tareas consumen tiempo y recursos que no se utilizan directamente para realizar los cálculos. Además, el sistema operativo necesita administrar los procesos, y todos pueden competir por recursos como el procesador y la memoria. Por esta razón, duplicar el número de procesos no significa necesariamente reducir el tiempo de ejecución a la mitad.

Otro factor importante es la ley de Amdahl, que explica que una parte de un programa puede ejecutarse en paralelo, mientras que otra parte debe realizarse de manera secuencial. Esta parte secuencial limita la aceleración máxima que se puede alcanzar, sin importar cuántos procesos se utilicen. En el caso de estimate_pi, aunque los cálculos se pueden repartir entre varios procesos, todavía existen tareas que requieren coordinación y que impiden aprovechar por completo todos los recursos disponibles.

En conclusión, la aceleración no es perfectamente lineal porque existen tareas adicionales de administración, comunicación y sincronización, además de una parte del programa que no se puede paralelizar por completo. Los resultados muestran que utilizar varios procesos reduce el tiempo de ejecución, pero llega un punto en el que agregar más procesos aporta muy poca mejora. En este experimento, utilizar 4 procesos ofrece una aceleración de 2.30 veces, mientras que utilizar 8 solo la aumenta a 2.39 veces. Esto demuestra que el rendimiento no depende únicamente de la cantidad de procesos, sino también de qué tan eficientemente se distribuye el trabajo y de los recursos disponibles en el equipo.

Ejercicio 2: Análisis algorítmico (Greedy Knapsack)

Indicación. Modificar el algoritmo voraz de la mochila para priorizar los objetos de menor peso, comparar ambos métodos y diseñar una instancia pequeña donde el criterio de menor peso obtenga un resultado peor que el de valor/peso.

La modificación consiste en cambiar la clave de ordenamiento del algoritmo voraz: en lugar de ordenar de mayor a menor razón v/w, se ordena de menor a mayor peso w. Después se recorren los objetos en ese orden y se agrega cada uno si cabe en la mochila.

Instancia diseñada

Capacidad W = 50. Objetos: A (v = 10, w = 10, razón 1), B (v = 100, w = 20, razón 5) y C (v = 120, w = 30, razón 4).

Resultados

Método	Objetos seleccionados	Capacidad usada	Valor total
Valor/peso	{B, C}	50 de 50	220
Menor peso primero	{A, B}	30 de 50	110
Óptimo (programación dinámica)	{B, C}	50 de 50	220

Tabla 2. Comparación de los dos criterios voraces con el óptimo.

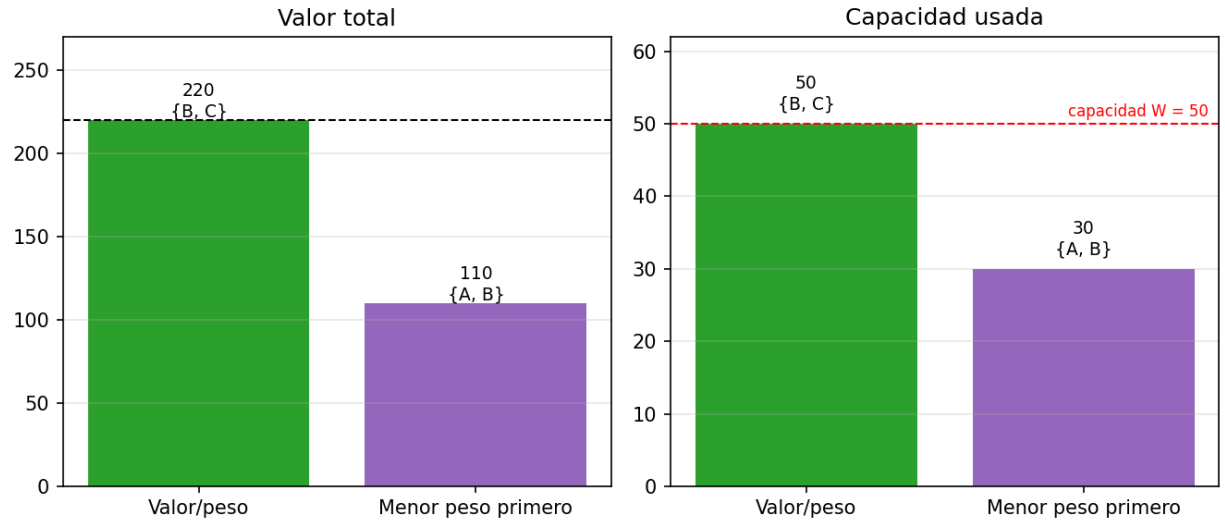


Figura 2. Valor total y capacidad usada con cada criterio.

Análisis

El criterio de menor peso toma A (w = 10) y B (w = 20) porque son los más ligeros. Con 30 de capacidad usada, C (w = 30) ya no cabe, y el valor total es 110. El criterio valor/peso toma B y C, llena la mochila (peso 50) y obtiene 220, que coincide con el óptimo. El criterio de menor peso ignora cuánto valor aporta cada unidad de peso, y por eso pierde 110 unidades de valor (50 %) en esta instancia. Además, deja 20 unidades de capacidad sin usar, que no puede ocupar con ningún objeto restante.

Cabe aclarar que valor/peso tampoco garantiza el óptimo en la mochila 0/1 (que es NP-difícil); solo es óptimo en la versión fraccionaria. En esta instancia sí lo alcanza.

Ejercicio 3: Búsqueda local (2-opt para TSP)

Indicación. Implementar la heurística 2-opt para el TSP y evaluar el número de mejoras necesarias para llegar al óptimo local desde (1) una solución inicial aleatoria y (2) una solución del Vecino Más Cercano iniciando en un nodo aleatorio.

La función two_opt_local_search recibe la matriz de distancias del grafo y un tour inicial. En cada paso evalúa todos los pares de aristas, aplica la inversión de segmento que más acorta el tour y cuenta una mejora; se detiene cuando ningún vecino mejora (óptimo local). Experimento: 100 instancias aleatorias de 30 ciudades; en cada una se parte de un tour aleatorio y de un tour del Vecino Más Cercano con nodo inicial aleatorio.

Resultados

Solución inicial	Mejoras (media / mediana)	Longitud inicial (media)	Longitud final (media / mediana)
Aleatoria	24.5 / 24	1553.6	474.2 / 470.7
Vecino Más Cercano	6.1 / 6	541.1	473.0 / 472.5

Tabla 3. Resultados de 2-opt en 100 instancias de 30 ciudades.

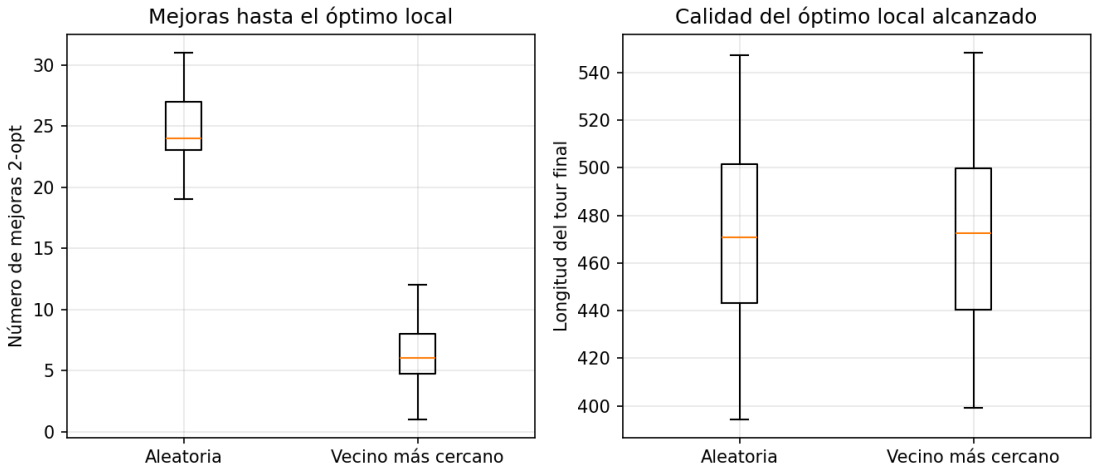


Figura 3. Diagramas de caja del número de mejoras y de la longitud del óptimo local alcanzado.

Análisis

Sí. En el diagrama de caja de la izquierda, la mediana de mejoras desde el inicio aleatorio es 24 contra 6 desde el Vecino Más Cercano, y las cajas ni siquiera se traslapan. Vecino Más Cercano necesitó menos mejoras que el inicio aleatorio en 100 de las 100 instancias. La razón es que el tour del Vecino Más Cercano ya es una solución buena (longitud media 541.1 contra 1553.6 del aleatorio), así que 2-opt tiene pocas aristas que cruzar o corregir.

En cuanto a la calidad, el diagrama de la derecha muestra que los óptimos locales alcanzados son muy parecidos (media 473.0 contra 474.2; el Vecino Más Cercano fue mejor en 51 de 100 instancias, prácticamente un empate). Por lo tanto, partir del Vecino Más Cercano reduce el costo de llegar al óptimo local, pero no lo mejora de forma apreciable: en ambos casos la búsqueda queda atrapada en un óptimo local cuya calidad depende más de la instancia que del punto de partida.

Ejercicio 4: Ajuste de parámetros (Simulated Annealing)

Indicación. Implementar Recocido Simulado usando 2-opt como operador de movimiento sobre una solución inicial aleatoria del TSP, y evaluar distintos esquemas de enfriamiento.

Se usa una instancia de 40 ciudades. La iteración 1 es el tour aleatorio inicial (longitud 2236.3); en cada iteración siguiente se propone un movimiento 2-opt aleatorio, se calcula ΔE = L_actual − L_nuevo y se acepta si ΔE > 0 o, si empeora, con probabilidad e^(ΔE/T). Todos los esquemas parten de T0 = 15 y terminan en Tf = 0.05:

- **Lineal:** T(t) = T0 − (T0 − Tf) · t/N.
- **Exponencial (lento):** T(t) = T0 · (Tf/T0)^(t/N), llega a Tf justo al final.
- **Exponencial (rápido):** el mismo decaimiento exponencial, pero comprimido para llegar a Tf en el 5 % de las iteraciones; el resto del tiempo T es casi cero.

Para el tiempo y la calidad final se repitió cada esquema con presupuestos de 20k, 50k, 100k, 200k, 400k iteraciones, 5 corridas por punto con semillas distintas. Como referencia, la búsqueda local 2-opt desde el mismo tour inicial llega a 537.2.

Resultados

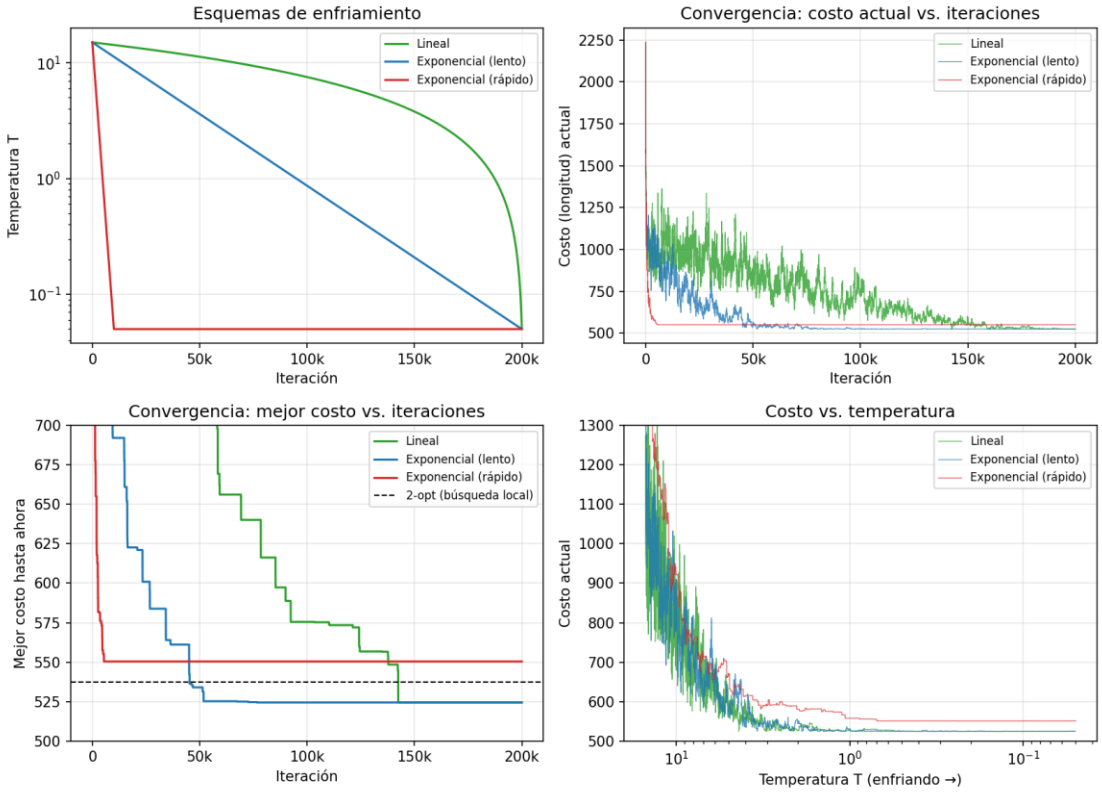


Figura 4. Curvas de convergencia: costo contra iteraciones y contra temperatura (una corrida de 200,000 iteraciones por esquema).

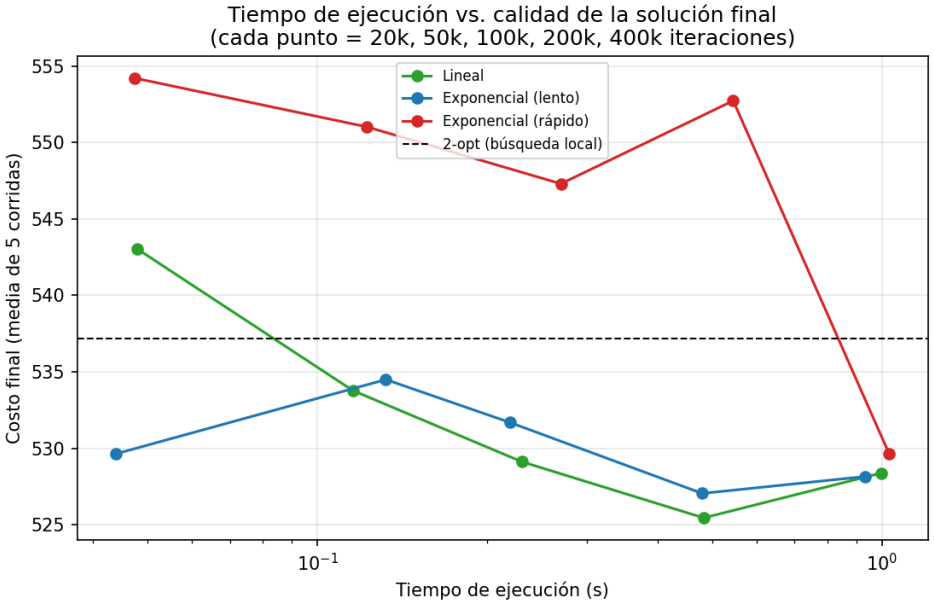


Figura 5. Tiempo de ejecución contra calidad de la solución final (media de 5 corridas).

Costo final medio	20k it.	50k it.	100k it.	200k it.	400k it.
Lineal	543.0	533.8	529.1	525.4	528.4
Exponencial (lento)	529.6	534.5	531.7	527.0	528.1
Exponencial (rápido)	554.2	551.0	547.3	552.7	529.6

Tabla 4. Costo final medio (longitud del tour) según el esquema y el número de iteraciones.

Tiempo de ejecución (s)	20k it.	50k it.	100k it.	200k it.	400k it.
Rango entre esquemas	0.04 – 0.05	0.12 – 0.13	0.22 – 0.27	0.48 – 0.55	0.93 – 1.03

Tabla 5. Tiempo de ejecución (mínimo y máximo entre los tres esquemas).

Análisis

Tiempo de cómputo. Los tres esquemas cuestan prácticamente lo mismo: el tiempo crece de forma lineal con el número de iteraciones (de unos 0.04 s con 20k iteraciones a unos 1.03 s con 400k) y las diferencias entre esquemas son de ruido de medición. Cada iteración hace el mismo trabajo (un movimiento 2-opt y un cálculo de ΔE); el esquema de enfriamiento solo cambia la temperatura usada en la regla de aceptación. Un esquema "más lento" solo cuesta más si se le asignan más iteraciones.

Calidad de la solución. Con 100,000 iteraciones o más, el esquema lineal (costo medio entre 525.4 y 529.1) y el exponencial lento (entre 527.0 y 531.7) dan soluciones mejores que la búsqueda local (537.2), porque a temperaturas intermedias aceptan movimientos peores y escapan de óptimos locales. Entre ellos la diferencia es pequeña frente a la variación entre corridas, así que con estos datos no puede declararse a uno claramente superior. El exponencial rápido se congela pronto: con 100k iteraciones su costo medio es 547.3 y con 200k es 552.7, peor que la búsqueda local, y su dispersión es la mayor (de 524.4 a 563.0 con 100k). Cuando T

cae a casi cero muy temprano, el algoritmo se comporta como subir colinas y queda atrapado en el óptimo local en el que haya caído.

Curvas de convergencia. En la figura 4, el esquema rápido baja el costo más rápido al inicio pero se estanca en un valor mayor; el exponencial lento y el lineal mantienen fluctuaciones fuertes mientras T es alta (diversificación) y las reducen al enfriar (intensificación), alcanzando los mejores costos en la segunda parte de la corrida. En la gráfica de costo contra temperatura se ve que el costo actual solo se estabiliza cuando T baja de aproximadamente 1.

Efecto de las iteraciones. El exponencial rápido con 400k iteraciones (529.6) coincide con el exponencial lento con 20k (529.6): ambos enfrían a la misma velocidad por iteración (en 20,000 iteraciones) y las iteraciones adicionales a temperatura casi cero ya no aportan. Lo que importa es cuántas iteraciones se pasan a temperaturas intermedias, no el total.

Ejercicio 5: Razón de aproximación (MST Approximation)

Indicación. Ejecutar el algoritmo de aproximación basado en MST sobre 100 instancias aleatorias de 9 ciudades del TSP, graficar la razón de aproximación y comparar el costo contra MST + 2-opt, la búsqueda local y el recocido simulado.

Para cada instancia se calcula el árbol de expansión mínima (Prim), el tour en preorden (que "acorta" los recorridos repetidos con la desigualdad triangular) y el óptimo exacto con Held-Karp. Sobre el tour del MST se aplica 2-opt como postprocesamiento. Para comparar, en las mismas instancias se ejecutan la búsqueda local 2-opt desde un tour aleatorio (ejercicio 3) y el recocido simulado con enfriamiento exponencial y 20,000 iteraciones (ejercicio 4). En las 100 instancias se verificó que w(T) ≤ Óptimo ≤ L(MST + 2-opt) ≤ L(MST) ≤ 2·w(T) ≤ 2·Óptimo.

Resultados

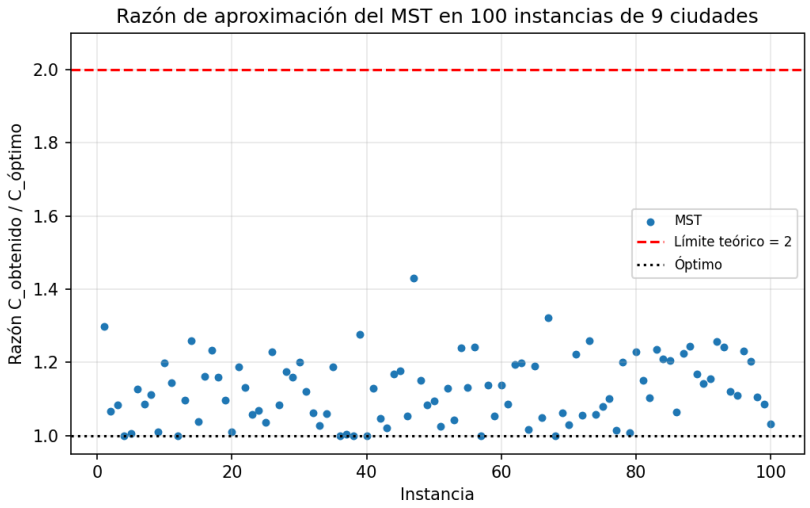


Figura 6. Dispersión de la razón C_obtenido / C_óptimo del MST, con el límite teórico de 2.

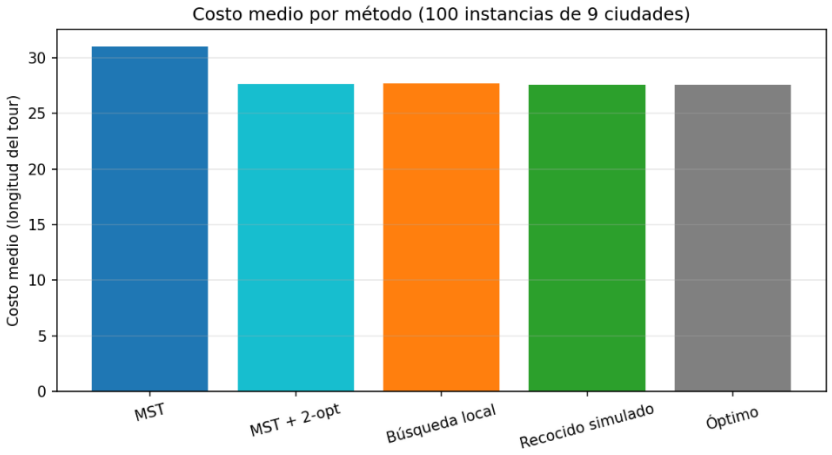


Figura 7. Costo obtenido por MST, MST + 2-opt, búsqueda local, recocido simulado y óptimo.

Método	Costo medio	Razón media	Razón máxima	Llega al óptimo	Tiempo (ms)
MST	31.01	1.124	1.431	7 / 100	0.03
MST + 2-opt	27.60	1.002	1.054	87 / 100	0.06
Búsqueda local (inicio aleatorio)	27.66	1.004	1.079	83 / 100	0.05
Recocido simulado	27.55	1.000	1.000	100 / 100	48.28
Óptimo (Held-Karp)	27.55	1.000	1.000	100 / 100	1.04

Tabla 6. Comparación de métodos en 100 instancias de 9 ciudades (tiempo por instancia).

Análisis

¿Qué tan cerca está la razón del límite de 2? Está lejos. La razón media del MST es 1.124 y la máxima observada es 1.431; ninguna instancia superó 1.5 y en 7 de las 100 el MST ya dio el óptimo. El histograma se concentra entre 1.0 y 1.3. Además, el peso del árbol representa en promedio el 70.6 % del óptimo, consistente con $w(T) \leq$ Óptimo. El límite de 2 es una garantía de peor caso: exige instancias construidas para que el recorrido doble del árbol no se pueda acortar con los atajos, algo que casi nunca ocurre con puntos aleatorios en el plano.

Impacto de aplicar 2-opt al resultado del MST. Mejora mucho: la razón media baja de 1.124 a 1.002, la máxima de 1.431 a 1.054, y las instancias en las que se llega al óptimo suben de 7 a 87. Como 2-opt nunca empeora el tour, se conserva la garantía de $\rho = 2$. Frente a los experimentos anteriores, MST + 2-opt (costo medio 27.60) es ligeramente mejor que la búsqueda local desde un tour aleatorio (27.66; óptimo en 83 instancias), de modo que un buen punto de partida lleva a óptimos locales de igual o mejor calidad. El recocido simulado obtiene el óptimo en las 100 instancias, pero con un tiempo por instancia (48.3 ms) cientos de veces mayor que el de MST + 2-opt (0.06 ms). Así, MST + 2-opt ofrece una calidad casi óptima con una garantía teórica y un costo mínimo, mientras que el recocido simulado compra la última fracción de calidad con mucho más cómputo. (Con 9 ciudades, el método exacto Held-Karp todavía es barato; su costo crece como $O(n^2 2^n)$.)

Anexo: Repositorio (GitHub)

https://github.com/EduardoRdzR/Ejercicios-Algoritmia-

Anexo: Video Demo

https://youtu.be/PKlBjAMoXW4